

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.DOI

Full Precision Hardware Implementation of the AlphaEvolve 4×4 Complex Valued Matrix Multiplication Algorithm

ETTORE NAPOLI¹, (Senior Member, IEEE)

¹Electrical Engineering and Applied Mathematics, Fisciano, SA, 84084 Italy e-mail: (enapoli@unisa.it).

Corresponding author: E. Napoli (e-mail: enapoli@unisa.it).



Finanziato
dall'Unione europea
NextGenerationEU

PNRR-POC-2022 12376833. Project CAREMODE. Funded by the EU. Next Generation EU. PNRR M6C2. 2.1
Improvement of the biomedical research of the SSN. CUP: D43C22005080006.

ABSTRACT Matrix multiplication is a fundamental operation across many domains. For 4×4 matrices, the naive algorithm requires 64 multiplications, while Strassen's method reduces this to 49. Recently the AlphaEvolve framework, an AI-driven approach for discovering optimized algorithms, introduced a groundbreaking 4×4 complex valued multiplication method requiring 48 multiplications, representing the current minimum. This work presents the first full precision, VLSI optimized hardware implementation of the AlphaEvolve algorithm for 4×4 complex matrix multiplication. The design achieves exact results without loss of numerical accuracy and is benchmarked against a naive and a Strassen based hardware implementation while varying the input bit width. The proposed circuits, synthesized in a 14 nm FinFET standard cell library demonstrates that, for operand widths up to 24 bits, the AlphaEvolve implementation generally incurs higher power, delay, and area costs than the naive implementation. When 32 bits or higher precision is required the proposed implementation of the AlphaEvolve algorithm achieves an increasing area reduction with respect to the naive implementation that reaches 31% for the 64 bits case. When compared with the Strassen based implementation the AlphaEvolve circuit achieves an area reduction for 48 bits or higher precision with a 9% reduction provided at 64 bits of precision.

INDEX TERMS Hardware architecture, matrix multiplication, VLSI, general matrix multiplication, AlphaEvolve algorithm, Strassen's algorithm.

I. INTRODUCTION

HARDWARE acceleration of general matrix multiplication (GEMM) plays a central role in modern computing that often relies on linear algebra operations, including its complex valued variants. Its applications span digital signal processing [1], convolutional neural networks [2]–[6], transformer architectures with attention and feed forward layers [7]–[9], quantum computing simulators [10], and MIMO wireless communication systems for channel estimation, detection, and precoding [11].

Many research efforts in GEMM have focused on hardware implementations of the naive matrix multiplication algorithm across various architectures [12]–[15]. In parallel, other studies have explored algorithms with inherently lower computational complexity. The most well known example is Strassen's algorithm, introduced in 1969, which demonstrated that the naive approach, requiring n^3 multiplications

to compute the product of two $n \times n$ matrices, is not optimal. In the specific case of 4×4 matrices, Strassen's method reduces the number of multiplications to 49 [16], at the expense of an increased number of add/subtract operations. The Strassen's algorithm has been improved, reducing the number of add/subtract operations while maintaining the number of multiplications, by Winograd in 1971 [17] and further optimized in 2017 and 2023 [18], [19]. Although the reduction in the number of multiplications provided by these algorithms is a positive phenomenon, the execution of these algorithms on general purpose central processing units (CPUs) and graphics processing units (GPUs) has been shown to be not suitable for achieving the promised theoretical speedups [20]–[22] mostly due to the increase in addition operations, to the complex and random-like access to the data required for processing and to the consequent complication of the computational process control.

However, in modern days some important aspects make these algorithms worth studying and revisiting.

- i) The growing use of custom accelerators, where abundant silicon area and fixed datapaths make it practical to handle the extra additions and irregular data flow of Strassen-type algorithms without the overheads seen on CPUs and GPUs.
- ii) The availability of wide on-chip datapaths, which keep all intermediate values local and avoid the costly memory accesses that could negate the benefits of reduced multiplications.
- iii) The importance of optimizing a fairly large computational block such as a 4×4 matrix multiplication, which can be implemented as an atomic accelerated operator in a larger system.
- iv) The shift toward Domain-Specific Architectures that use specialized primitives for complex-valued arithmetic to handle 5G/6G communication standards.

Motivated by these developments, several recent works have revisited Strassen-like methods through custom hardware architectures specifically designed to exploit their structure, as shown in [23]–[25].

In 2025, the AlphaEvolve framework introduced a groundbreaking algorithm specifically designed for complex valued 4×4 matrix multiplication, [26]. By employing AI-driven algebraic transformations, this algorithm reduces the number of multiplications to 48, improving upon the previous best known methods such as Strassen's algorithm.

A reduction of one multiplication operation is extremely interesting from a theoretical point of view but requires an in depth research effort to determine which type of advantage, if any, this can provide in a circuitual implementation. As an example, since the area of binary multipliers grows quadratically with bit width while additions grow linearly, reducing the number of multiplications should yield greater benefits as bit width increases. However, the break even point is uncertain and may be so large as to limit practical usefulness.

This paper proposes, to the best of the author's knowledge, the first circuitual implementation of the AlphaEvolve algorithm.

The complex valued 4×4 AlphaEvolve matrix multiplication algorithm is fully unrolled and implemented in GlobalFoundries 14 nm FinFET standard cell VLSI technology. The proposed implementation is full precision providing no loss of significant digits during the computation and is completely combinational to eliminate data bottlenecks from the assessment of the performance of the algorithms. It is worth highlighting that converting the theoretical formulation provided by the AlphaEvolve framework into a practical VLSI hardware implementation entails numerous non-obvious design decisions that directly influence the final power, area, and delay characteristics of the circuit.

The proposed implementation is compared against a hardware implementation of the naive algorithm that uses 64 complex multiplications and against Strassen based hardware

implementation that reduces the number of complex multiplications to the previously known limit of 49. These reference designs are intentionally realized using straightforward HDL descriptions and conventional complex multiplication schemes, so as to serve as robust baselines. In contrast, the AlphaEvolve implementation adopts the Katsabura multiplication method to highlight its key advantage, namely the reduction in the number of multiplications. Therefore, the comparison should be interpreted as an initial and exploratory result aimed at exposing the potential of the proposed algorithm, rather than as a fully homogeneous architectural benchmarking.

The motivation of the presented research activity is:

- provide the first circuit-level implementation of the AlphaEvolve algorithm, enabling a concrete performance baseline for future research;
- assess whether the algorithmic reduction in multiplications translates into measurable hardware performance gains;
- analyze how the performance advantage (or disadvantage) scales with operand bit-width;
- release an open-source implementation to support reproducibility and further investigations.

The paper is organized as follows. Section II reviews the Rank-48 AlphaEvolve algorithm. Sections III and IV present the methodology used to derive the corresponding hardware architecture. Section V details the implementation of the proposed circuit, while Section VI describes the naive design and the Strassen based implementations used as a reference. Section VII reports the electrical performance comparison, and Section VIII concludes the paper.

II. THE ALPHAEVOLVE ALGORITHM

Assume that A and B are 4×4 complex valued matrices and we want to compute $C = A \cdot B$, where C is also a 4×4 complex valued matrix. The naive algorithm given in (1) has a rank of 64 since it requires 64 complex multiplications. It also needs 48 complex additions.

$$C_{i,j} = \sum_{k=1}^4 A_{i,k} B_{k,j}, \quad 1 \leq i, j \leq 4. \quad (1)$$

The complex valued 4×4 AlphaEvolve matrix multiplication rank 48 factorization algorithm (uses 48 multiplications, one less than the best known algorithms, [16]–[19]) is described in tensor notation. It is presented in [26] and detailed at [27]. A verification and test environment of the matrix multiplication can be found at [28].

A. TENSOR NOTATION OF THE ALPHAEVOLVE ALGORITHM

The AlphaEvolve rank 48 decomposition of the 4×4 matrix multiplication is performed over the complex field $0.5\mathbb{C}$. Hence, the scalar coefficients appearing in the factorization are restricted to values 0.5 , $0.5j$, or compositions thereof.

This choice of coefficients simplifies the arithmetic by limiting multiplications to fixed scalars involving halves and imaginary halves.

The decomposition is expressed in tensor notation as:

$$T = \sum_{r=1}^R u^{(r)} \otimes v^{(r)} \otimes w^{(r)}, \quad (2)$$

where:

- T is the $16 \times 16 \times 16$ matrix multiplication tensor (where 16 represents the 16 elements of a flattened 4×4 matrix). The tensor represents the matrix multiplication operation and is the same for every algorithm.
- $\otimes$ denotes the outer product, that multiplies every element of the matrix (or vector) on the left by every element of the matrix (or vector) on the right.
- $R = 48$ is the tensor rank, i.e., the number of multiplications used. This number depends on the algorithm. The naive implementation would have $R = 64$ while Strassen's or Winograd's algorithms would have $R = 49$.
- u, v, w are 48×16 matrices with entries in:

$$\left\{ 0, \pm \frac{1}{2}, \pm \frac{1}{2}j, \pm \left(\frac{1}{2} + \frac{1}{2}j \right), \pm \left(\frac{1}{2} - \frac{1}{2}j \right) \right\}$$

- The $u^{(r)}$, $v^{(r)}$, and $w^{(r)}$ terms are the 1×16 vectors corresponding to the r -th row of the matrix. These matrices are the core of the algorithm discovered by AlphaEvolve and differ in each algorithm. The matrices for the AlphaEvolve algorithm can be found at [27].

Intuitively, the vectors $u^{(r)}$ and $v^{(r)}$ act as selection masks that determine which elements of matrices A and B are combined in the r -th multiplication. The vector $w^{(r)}$ then is the map that specifies where that specific product is accumulated in the resulting matrix C . By repeating this 48 times and summing the results, the full matrix multiplication is completed.

B. STEP BY STEP EXECUTION OF THE ALGORITHM

The computation to multiply the 4×4 complex valued matrices A and B proceeds as follows:

- **Step 1:** For each $r \in \{1, \dots, 48\}$, compute the Frobenius inner products, denoted as $\langle \cdot, \cdot \rangle$, between the vector of coefficients and the matrix entries:

$$a_r = \langle u^{(r)}, A \rangle \quad b_r = \langle v^{(r)}, B \rangle \quad (3)$$

This step computes the 48 a_r scalars, and the 48 b_r scalars, which represent linear combinations of the elements of matrices A and B , respectively. No multiplications are required since the coefficients of u, v belong to $0.5\mathbb{C}$ allowing the computation of a_r and b_r using only additions and subtractions (since the bit shift for the 0.5 constant is fixed and can be applied directly to the output C matrix).

- **Step 2:** For each $r \in \{1, \dots, 48\}$, multiply the resulting a_r and b_r scalars to obtain the 1×48 vector m whose individual elements are:

$$m_r = a_r \cdot b_r, \quad (4)$$

this step requires R complex multiplications.

- **Step 3:** Combine the results using $w^{(r)}$ to get:

$$C = \sum_{r=1}^{48} m_r w^{(r)} = m w \quad (5)$$

this step, driven by the w matrix, computes 16 linear combinations of the elements of the vector m to finally obtain the elements of the product matrix C . It involves sums and subtractions and no multiplications since the elements of w belong to $0.5\mathbb{C}$.

III. SCALAR FORM OF THE ALPHAEVOLVE ALGORITHM

The tensor representation of the algorithm is not suited for hardware implementation and therefore requires transformations that enable a circuit level realization. The transformations are not unique and influence the final performances.

In this paper, the applied transformations are: the expansion of the Frobenius products and of the $m w$ operation into linear combinations of the input matrices (Section III-A); the use of the Katsabura algorithm for the 48 complex multiplications (Section III-B).

A. EXPANDING THE FROBENIUS AND THE INNER PRODUCTS

Step 1 of the AlphaEvolve algorithm computes the Frobenius products, producing the complex terms a_r and b_r , which we re-index as $a_0 \dots a_{47}$ and $b_0 \dots b_{47}$, expressed as linear combinations of the A and B matrices. Step 3 computes linear combinations of the multiplication results, m_r , to obtain the output matrix C .

Below are some examples of the linear combinations, where all terms are complex numbers, and $A_{i,j}$, $B_{i,j}$, and $C_{i,j}$ with $i, j \in \{0, \dots, 3\}$ denote individual matrix elements. The detailed equations are provided in tensor form at [27] and fully expanded at [28].

$$a_0 = (0.5 + 0.5j)A_{0,0} + \dots + (0.5 - 0.5j)A_{3,1} \quad (6)$$

$$a_{47} = 0.5A_{0,0} + 0.5jA_{0,1} + \dots + 0.5A_{3,3} \quad (7)$$

$$b_0 = -0.5B_{0,0} - 0.5B_{1,0} + 0.5B_{2,0} - 0.5jB_{3,0} \quad (8)$$

$$b_{47} = 0.5B_{0,0} + 0.5B_{1,1} + 0.5B_{2,1} + 0.5jB_{3,0} \quad (9)$$

$$C_{0,0} = 0.5jm_0 - 0.5jm_1 + \dots + 0.5m_{47} \quad (10)$$

$$C_{3,3} = -0.5jm_0 - 0.5jm_1 + \dots - 0.5m_{47} \quad (11)$$

In total there are 112 complex expressions (96 for the two Frobenius products and 16 for the inner product).

The first transformation consists in doubling all the equations to remove the common $\frac{1}{2}$ term. Each doubling will be later recovered by introducing a division step at the end of the processing. For the sake of simplicity, from now on the

notation $a_0 \dots a_{47}$ and $b_0 \dots b_{47}$ will indicate Eqs. (6)–(11) multiplied by 2. Note that multiplying a and b terms by two determines a multiplication by 4 of the term m ; together with the multiplication by 2 applied to Eqs. (10) and (11), results in each element of C being multiplied by 8. This factor of 8 is compensated in the final circuit by a 3-bit right shift, as shown in Section V-B.

Afterwards, the equations are expanded to isolate the expressions for the real and imaginary parts of each term, leading to a total of 224 real multi operand add/subtraction expressions that are finally suitable for hardware implementation.

As an example, the equations below show the multi operand expressions to compute the real and the imaginary part of a_0 . The complete set of 224 equations, not reported in any AlphaEvolve related publication, is clearly too long to fit in the length of the paper, and is made available at [29] together with the HDL code of the circuit and the test bench to ensure research reproducibility and further investigations.

$$\begin{aligned} \Re\{a_0\} = & \Re\{A_{0,0}\} - \Im\{A_{0,0}\} + \Re\{A_{1,0}\} - \Im\{A_{1,0}\} + \\ & + \Re\{A_{0,1}\} + \Im\{A_{0,1}\} + \Re\{A_{1,1}\} + \Im\{A_{1,1}\} + \\ & + \Re\{A_{0,2}\} + \Im\{A_{0,2}\} + \Re\{A_{1,2}\} + \Im\{A_{1,2}\} + \\ & + \Re\{A_{0,3}\} + \Im\{A_{0,3}\} + \Re\{A_{1,3}\} + \Im\{A_{1,3}\} \end{aligned} \quad (12)$$

$$\begin{aligned} \Im\{a_0\} = & \Re\{A_{0,0}\} + \Im\{A_{0,0}\} + \Re\{A_{1,0}\} + \Im\{A_{1,0}\} + \\ & - \Re\{A_{0,1}\} + \Im\{A_{0,1}\} - \Re\{A_{1,1}\} + \Im\{A_{1,1}\} + \\ & - \Re\{A_{0,2}\} + \Im\{A_{0,2}\} - \Re\{A_{1,2}\} + \Im\{A_{1,2}\} + \\ & - \Re\{A_{0,3}\} + \Im\{A_{0,3}\} - \Re\{A_{1,3}\} + \Im\{A_{1,3}\} \end{aligned} \quad (13)$$

B. KATSABURA ALGORITHM FOR COMPLEX MULTIPLICATIONS

The second step of the AlphaEvolve algorithm requires 48 complex multiplications: $m_0 = a_0 \cdot b_0, \dots, m_{47} = a_{47} \cdot b_{47}$.

The circuit proposed in this paper implements the complex multiplications using the Katsabura algorithm that performs each complex multiplications using 3 real multiplications and 5 additions instead of using the conventional method that uses 4 real multiplications and 2 additions.

The total number of real multiplications required by the proposed circuit is then $3 \cdot 48 = 144$.

The Katsabura algorithm multiplies $(a + jb)$ and $(c + jd)$ computing the intermediate products:

$$P_1 = a \cdot c, \quad P_2 = b \cdot d, \quad P_3 = (a + b) \cdot (c + d),$$

and combines them to get the real and the imaginary parts:

$$\Re[(a + jb) \cdot (c + jd)] = a \cdot c - b \cdot d = P_1 - P_2$$

$$\Im[(a + jb) \cdot (c + jd)] = a \cdot d + b \cdot c = P_3 - P_1 - P_2.$$

IV. COMMON SUBEXPRESSIONS ELIMINATION

Equations (12), (13) together with those reported at [29] are now suited for hardware implementation but can be further optimized since many terms are shared across the 224 multi operand add/subtract expressions. As is well known in compiler design and High Level Synthesis, such redundancies

can be reduced using a Common Subexpression Elimination (CSE) algorithm.

The specific algorithm applied in this paper is detailed in Algorithm 1.

Algorithm 1 Common Subexpression Elimination

Require: Initial list of operands: $+\Re(A_{0,0})$, $-\Im(A_{2,1})$, $+\Re(B_{3,3})$, etc.

Require: List of expressions to be optimized

```

1: while true do
2:   for each expression do
3:     Define subexpressions combining all the pairs of
       operands
4:     Store the subexpressions and count how many times
       each subexpression appears
5:   end for
6:   Find the subexpressions that have the maximum num-
       ber of occurrences
7:   if the maximum number of occurrences is one then
8:     break
9:   end if
10:  Pick one random subexpression among those that have
       the maximum number of occurrences
11:  Define a new expression that computes the picked
       maximum
12:  Append the new expression to the list of expressions
       to be optimized
13:  Substitute the new expression in previous expressions
14:  Define a new operand associated to the new expression
15:  Append the new operand to the list of operands
16: end while

```

In short, the algorithm iteratively builds the list of all the subexpression formed by adding or subtracting two operands, counts their occurrence, picks the subexpression that is most used (if more than one chooses randomly), and computes this subexpression separately.

Unfortunately, CSE algorithms do not guarantee the optimal solution and this is also true for Algorithm 1. However, Algorithm 1 randomly picks which subexpression to isolate in step 10, hence it can be run multiple times to generate a range of solutions increasing the chance of approaching the optimum.

In this paper the algorithm has been run on the set of expressions that define $a_0, \dots, a_{47}$, $b_0, \dots, b_{47}$, and $C_{0,0}, \dots, C_{3,3}$. To better optimize the expressions, the Algorithm 1 has been run 10^4 times searching for the result with the minimum number of add/subtract operations for each of the three cases. Algorithm 1 is able to reduce the number of operations in every run, but with a spread that justifies the 10^4 runs indicated above. As an example, when applied to the set of expressions for $C_{0,0}, \dots, C_{3,3}$, the algorithm starts from an initial count of 992 add/subtract operations. The distribution of the resulting number of operations is approximately Gaussian, with an average of 510 operations and a standard deviation of 5. After 10^4 runs, some less

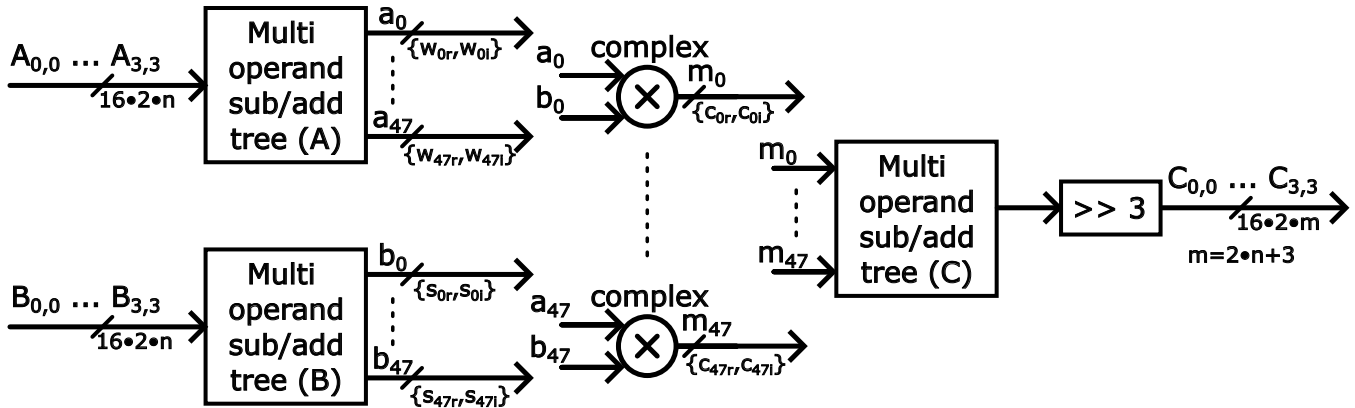


FIGURE 1: Circuit schematic of the proposed circuit for 4×4 complex valued matrix multiplication using the rank 48 AlphaEvolve algorithm. The bit width of the elements of the input matrices is n .

favorable runs with more than 525 operations have been observed, while only a few runs achieve results close to the minimum of 493 operations. The resulting expressions are open sourced at [29]. It is worth highlighting that these are still not guaranteed to be optimal and further research could reduce the required number of operations or other performance parameters of the implementation.

Using Algorithm 1, the expressions for $a_0, \dots, a_{47}$ have been simplified reducing the number of add/subtract operations from 1408, that are required with a straightforward implementation of the equations reported in the AlphaEvolve Colab [27], to 412. Similar results have been obtained applying Algorithm 1 to the expressions for $b_0, \dots, b_{47}$ and $C_{0,0}, \dots, C_{3,3}$ whose number of add/subtract operations has been reduced from 688 and 992 to 291 and 493, respectively.

V. HARDWARE IMPLEMENTATION OF THE ALPHAEVOLVE ALGORITHM

A. MEMORY ARCHITECTURE

The performance of a matrix multiplication circuit is heavily dependent on the memory architecture and data-path width. As pointed out in [25], the execution of the theoretically high performance Strassen's and Winograd's algorithms on general purpose CPUs and GPUs has not provided the algorithm's promised theoretical speedups also due to data movement bottlenecks.

In this paper the circuit design of the AlphaEvolve algorithm has been fully parallelized and implemented as a combinatorial datapath following the schematic in Fig. 1. In other terms, the design assumes that all the data are available in parallel and that there is no speed constraint that requires the use of pipelining techniques. This choice is motivated by two considerations:

- modern processing architectures provide wide on-chip datapaths, which allow keeping all intermediate values locally avoiding costly memory accesses;
- the circuit is intended as a custom accelerator implementing an atomic 4×4 matrix multiplication operation,

which is repeatedly invoked in computations involving larger matrices.

The fully parallelized design requires a datapath of $16 \cdot 2 \cdot n$ bits (16 complex elements on n bits) for each of the input matrices and a datapath of $16 \cdot 2 \cdot (2 \cdot n + 3)$ bits for the output. For $n = 32$ the input and output datapaths are 2048 bits and 2144 bits, respectively. Although assuming all inputs are available in parallel may seem demanding, it is feasible in modern CPU architectures. As an example, the matrix oriented AMX instructions in x86 CPUs can easily store both 4×4 input matrices within a single tile, each of which holds 16×64 32 bits elements, for a total of 32768 bits.

B. CIRCUIT SCHEMATIC

The circuit implementing the AlphaEvolve algorithm follows the schematic in Fig. 1.

The circuit is composed of 48 Katsabura multipliers that compute $m_0, \dots, m_{47}$, three multi operand add/subtraction trees for the computation of $a_0, \dots, a_{47}$, $b_0, \dots, b_{47}$, and $C_{0,0}, \dots, C_{3,3}$, and the divide by 8 shift block at the right that balances the multiplications by two introduced in Section III-A.

The schematic reports the bit width of the input and output signals as a function of n , that is the number of bits of the real and imaginary numbers that compose the input matrices. The bit width of the internal nodes is indicated with symbols. As an example, for b_0 the bit width is indicated as $\{s_{0r}, s_{0i}\}$ for the real and the imaginary parts of the signal. A detailed explanation of how the bit-width was determined is nontrivial and is provided in Section V-C.

Each Katsabura multiplier has the structure shown in Fig. 2. It implements one complex multiplication using 3 real multiplications and 5 add/sub operations. The 48 Katsabura multipliers only differ in the number of bits of the input and of the output. For instance, Fig. 2, shows the multiplier implementing the complex multiplication $m_0 = a_0 \cdot b_0$ whose components are indicated as $m_{0r} + jm_{0i} = (a_{0r} + ja_{0i}) \cdot (b_{0r} + jb_{0i})$.

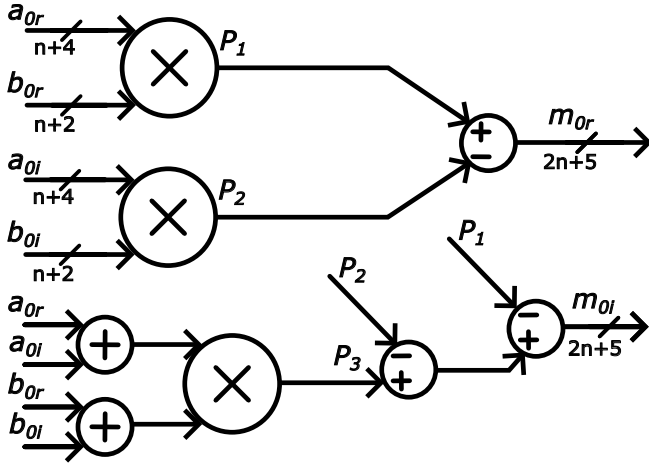


FIGURE 2: Detailed schematic of the Katsabura multiplier implementing $m_0 = a_0 \cdot b_0$. The bit width of the elements of the input matrices is n .

The multi operand add/subtraction trees implement Eqs. (6)–(11) and are composed solely of adders and subtractors. The add/subtraction tree indicated with (A) contains 412 add/sub circuits. The add/subtraction tree indicated with (B) contains 291 add/sub circuits, while the block that combines the outputs of the multipliers, indicated with (C), contains 493 add/sub circuits. A detailed schematic of the (A), (B), and (C) blocks would be very complex and is not reported here; further implementation details are left to the HDL circuit description discussed in Section V-D and fully available at [29].

C. PRECISION AND NUMERICAL STABILITY

The input data for both the implemented AlphaEvolve algorithm and the baseline naive algorithm consist of signed complex integers represented in two's complement on n bits. Accordingly, both the real and imaginary components take values in the range $[-2^{n-1}, 2^{n-1} - 1]$.

Each entry of the result is obtained as the sum of four products of complex numbers. To prevent overflow when summing these four partial products, the real and imaginary parts of the output are represented using $m = 2n + 3$ bits. A detailed explanation of how the number of output bits is derived is given in Appendix .

The circuit proposed in this paper provides exact results with no loss of precision. To achieve this, it is necessary to allocate the correct number of bits not only for the output but also for the internal signals of the circuit in Fig. 1. Furthermore, to obtain an optimized circuit, it is important to avoid using more bits than required.

Unfortunately, the bit width representation of the signals shown in Fig. 1, of the internal signals of the multi operand add/subtraction blocks, and of the intermediate signals of the Katsabura multipliers discussed in Section III-B and shown in Fig. 2, is difficult to derive in closed form. This is because, while the initial steps of the algorithm involve independent quantities whose output ranges can be easily determined,

the signals following the first add/subtraction stages are no longer independent, making their ranges difficult to predict. A clear example is shown in Fig. 2, where the bit width of the output signals is limited to $2n + 5$. If the input signals were independent and spanned their full range, the bit widths should instead be $2n + 7$ and $2n + 10$ for m_{0r} and m_{0i} , respectively. However, the theoretical worst-case does not apply to the architecture in Fig. 2 because the intermediate terms are not independent; for instance, P_2 and P_3 are both functions of a_{0i} . This correlation limits the actual dynamic range, allowing for the reduced bit-widths observed.

In this paper, a simulation based bit width dimensioning of the internal signals has been employed. A software model of the AlphaEvolve algorithm has been designed. Once fixed the number of bits of the inputs for the model, n , it has been run 3×10^4 times feeding it with matrices whose entries are randomly chosen to be the extremes of the allowed range¹ and the range of all the internal signals has been stored. This allowed to determine a reliable guess of the required bit width for all the internal signals for a given n value.

The simulation has been repeated other two times increasing the n value to obtain a linear dependence between n and the bit width of every signal. While two points theoretically define a line, using three values of n allowed to identify the very rare cases in which one of the simulations did not hit the extreme range for some of the signals. A subsequent manual check of residual cases in which the bit width requirements for a signal were not lying on a line has been conducted and in this case the worst case bit width has been chosen.

Illustrative case - Linear Fit. If a signal t required 7, 11, and 19 bits for input bit widths $n = \{4, 8, 16\}$, a bit width requirement of $n + 3$ has been derived.

Illustrative case - Underestimation Correction. If a signal q required 12, 19, and 36 bits for the same input widths, the middle value (19) was flagged as an underestimation caused by the random nature of the simulation. In this case, the more conservative requirement of $2n + 4$ was chosen to ensure hardware reliability.

As an example, the proposed method allowed to determine that the bit width for the imaginary part of b_0 , named s_{0i} in Fig. 1, is $n + 2$, while the bit width of the real part of m_{10} is $c_{10r} = 2n + 6$, and the largest signals are the output signals for four of the multipliers whose bit width is $2n + 8$.

The above described algorithm does not guarantee the optimal result and can also end by using too few bits on some signal causing overflow and a not correct output for the circuit. The final validation of the bit width dimensioning has been carried out by simulating the synthesized circuit using a test bench and checking the correctness of the outputs against a bit accurate software model of the matrix multiplication. The test bench applies approximately 3300 different test vectors (being A and B complex matrices this results in a total of 32 input values on n bits for each test vector), that

¹Extreme output values occur when inputs reach their maximum or minimum. The choice of extreme inputs is therefore increasing the chance of capturing the true extremes.

have been determined mixing corner cases, such as inputs that produce maximum or minimum values, small inputs, large magnitude elements differing by small amounts, trivial cases, such as multiplication by 0 or by identity matrices, and randomly generated values. The SystemVerilog test bench and the test vectors files for all the bit width of the circuit considered in this paper are available at [29].

D. HARDWARE DESCRIPTION

The circuit has been described in SystemVerilog using the bit widths obtained in Section V-C, together with the multi-operand add/subtraction blocks introduced in Section IV and the Katsabura multipliers described in Section III-B.

The HDL description is parameterized by n , the number of bits of the elements in the input matrices. The arithmetic operations of addition, subtraction, and multiplication are described using the Verilog operators $+$, $-$, and $*$, allowing the synthesizer to optimize the arithmetic based on the available technology.

Because the HDL implementation is extensive (over 2,000 lines of SystemVerilog), and to ensure completeness and reproducibility of the results, it is provided in full in the associated repository [29].

VI. IMPLEMENTATION OF THE REFERENCE DESIGNS FOR MATRIX MULTIPLICATION

This sections reports the implementation details for the naive implementation and the Strassen based implementation of the complex valued matrix multiplication circuit used as reference designs for the AlphaEvolve implementation.

A. IMPLEMENTATION OF THE NAIVE ALGORITHM FOR MATRIX MULTIPLICATION

The AlphaEvolve circuit for matrix multiplication has been compared against the naive implementation of the 4×4 complex valued matrix multiplication algorithm.

The naive algorithm implements the row by column multiplication shown in (1) and requires 64 complex multiplications and 48 complex additions. In this paper, in order to compare the AlphaEvolve algorithm with a basic approach, the complex multiplications in the naive algorithm have been implemented in the classical way, using four binary multiplications.

Moreover, also the naive algorithm has been implemented with the goal of maintaining full precision. Therefore, the bit widths of the internal signals and of the output matrix elements have been determined so as to avoid overflow and to prevent any loss of significant bits.

The circuit implementation of the naive algorithm has been described in SystemVerilog, and the source code, which can be used as a reference, is available at [29].

B. IMPLEMENTATION OF THE STRASSEN BASED MATRIX MULTIPLICATION CIRCUIT

The AlphaEvolve circuit for matrix multiplication has also been compared against a Strassen based implementation of

the 4×4 complex valued matrix multiplication algorithm.

The Strassen algorithm, originally proposed in [16], is based on a technique that multiplies 2×2 matrices using 7 multiplications and 18 additions/subtractions, achieving a reduction in computational complexity with respect to the naive algorithm, which requires 8 multiplications and 4 additions. The Strassen's algorithm applied to 2×2 matrices firstly combines the A and B terms and multiplies the results as in (14).

$$\begin{aligned} M_1 &= (A_{1,1} + A_{2,2}) \cdot (B_{1,1} + B_{2,2}) \\ M_2 &= (A_{2,1} + A_{2,2}) \cdot B_{1,1} \\ M_3 &= A_{1,1} \cdot (B_{1,2} - B_{2,2}) \\ M_4 &= A_{2,2} \cdot (B_{2,1} - B_{1,1}) \\ M_5 &= (A_{1,1} + A_{1,2}) \cdot B_{2,2} \\ M_6 &= (A_{2,1} - A_{1,1}) \cdot (B_{1,1} + B_{1,2}) \\ M_7 &= (A_{1,2} - A_{2,2}) \cdot (B_{2,1} + B_{2,2}) \end{aligned} \quad (14)$$

The $M_1, \dots, M_7$ terms are then combined to obtain:

$$\begin{aligned} C_{1,1} &= M_1 + M_4 - M_5 + M_7 \\ C_{1,2} &= M_3 + M_5 \\ C_{2,1} &= M_2 + M_4 \\ C_{2,2} &= M_1 - M_2 + M_3 + M_6 \end{aligned} \quad (15)$$

The Strassen's algorithm can be hierarchically exploited to multiply higher-order matrices. When applied to 4×4 complex matrices, the input matrices are first partitioned into four 2×2 submatrices. The Strassen's algorithm is then applied to the resulting 2×2 matrix whose elements are themselves 2×2 complex matrices, which are therefore multiplied 7 times. Each of these 2×2 complex matrix multiplications is again implemented using the Strassen method, requiring 7 complex multiplications per instance. Overall, the total number of complex multiplications is $7 \cdot 7 = 49$. The total number of complex addition/subtraction operations can be derived to be 198 and correspond to 396 real addition/subtraction operations.

In this paper the implementation of the circuit follows (14),(15). The elementary 2×2 multiplication block has been designed and tested separately. Due the simplicity of the operations, the bit width of the internal signals has been analytically determined so as to avoid overflow and to prevent any loss of significant bits. The complex multiplications have been implemented using four binary multiplications. As detailed in the introduction, this design choice is aimed at providing a robust baseline design for the comparison. The 2×2 multiplication block is then exploited as an elementary operator for the 4×4 multiplication block that also implements the Strassen's algorithm (14),(15) but with the arithmetic operations that operate on 2×2 matrices. Bit width sizing at the 4×4 hierarchical level is simplified by the availability of yet optimized 2×2 blocks and can be, also in this case, analytically derived to guarantee full precision of the result.

TABLE 1: Electrical performance of reference and AlphaEvolve designs. Percentages are the relative increase of AlphaEvolve over Naive.

#bits	Algorithm	Area	(%)	Power	(%)	Delay	(%)
		($\mu\text{m}^2 \times 10^4$)		(mW/MHz)		(ns)	
4	Naive	0.41	–	0.013	–	0.80	–
	Strassen	0.80	–	0.078	–	1.70	–
	AlphaEvolve	1.84	348 %	0.48	3615 %	2.53	216 %
8	Naive	1.44	–	0.086	–	1.71	–
	Strassen	1.98	–	0.25	–	2.37	–
	AlphaEvolve	3.53	145 %	1.14	1224 %	4.61	170 %
16	Naive	5.45	–	0.56	–	3.52	–
	Strassen	5.55	–	1.17	–	5.01	–
	AlphaEvolve	7.74	42 %	2.95	424 %	6.24	77 %
24	Naive	12.3	–	2.31	–	4.00	–
	Strassen	10.7	–	2.43	–	6.63	–
	AlphaEvolve	13.1	7 %	5.14	122 %	7.38	85 %
32	Naive	20.6	–	4.70	–	5.98	–
	Strassen	18.4	–	3.15	–	6.86	–
	AlphaEvolve	19.8	-4 %	9.82	109 %	8.00	34 %
48	Naive	50.7	–	14.1	–	6.57	–
	Strassen	38.4	–	9.71	–	8.19	–
	AlphaEvolve	38.0	-25 %	14.5	3 %	9.08	38 %
53	Naive	61.2	–	19.0	–	7.48	–
	Strassen	46.5	–	11.3	–	8.78	–
	AlphaEvolve	44.3	-28 %	17.5	-8 %	9.83	31 %
64	Naive	88.0	–	22.5	–	8.51	–
	Strassen	66.7	–	16.6	–	9.57	–
	AlphaEvolve	61.0	-31 %	24.2	8 %	9.99	17 %

The arithmetic operations are described using Verilog operators +, –, and *. The specific implementation of the adder trees and binary multiplication architectures, has been left to the synthesizer. This ensures the baseline remains reproducible and straightforward, allowing the tool to optimize based on timing constraints. The circuit implementation of the Strassen algorithm has been described in SystemVerilog, and the source code, which can be used as a reference, is available at [29].

VII. ELECTRICAL PERFORMANCE

The AlphaEvolve and the reference naive and Strassen matrix multiplication circuits, described in Sections V and VI, were implemented in a VLSI standard cell flow to evaluate electrical performance under varying input precisions (4, 8, 16, 24, 32, 48, 53, and 64 bits).

All designs were synthesized in GlobalFoundries 14 nm FinFET standard cell technology (0.8 V supply, regular V_T) using Cadence Genus Version 18.13 with a timing constraint of 10 ns. Post synthesis verification was performed by simulating the netlists with the test bench detailed in Section V-C and comparing outputs against the golden results obtained from a bit accurate software model.

Power dissipation was estimated by simulating the synthesized netlists at an input toggle frequency of 100 MHz, with outputs loaded by a NAND2X cell. Switching activity was collected over 10^3 random input vectors.

The electrical performance results are reported in Table 1.

In terms of silicon area occupation, Table 1 shows that for bit widths up to 24, the naive implementation requires less area than the proposed implementation of the AlphaEvolve algorithm (AE, from now on). For example, in the 8 bit case, the AE implementation requires more than twice the

area of the naive implementation. As the bit width increases, the impact of the larger number of multipliers in the naive design becomes dominant, since their area grows approximately with the square of the number of bits, and the gap between the two implementations decreases. For the 32 bit case, the area occupation is approximately equivalent, with AE requiring 4% less area than the naive implementation. It can be stated that at 32 bits the increased number of adders in the AE implementation is balanced by the larger number of multipliers of the naive implementation.

Increasing the bit width keeps widening the gap. For the 64 bit implementation, the AE version occupies 31% less area than the naive implementation, which requires 0.88 mm^2 compared to the 0.61 mm^2 of the AE implementation.

The silicon area occupation comparison against the Strassen based circuit implementation follows a similar trend. At 8 bit the AE circuit is 80% larger than the Strassen based circuit but the difference reduces increasing the bit width. At 32 bit the difference in area occupation is very small but still favors the Strassen based implementation but the AE circuit has an advantage for the 48, 53, and 64 bit implementations. The improvement in this case is smaller and at 64 bit the AE circuit occupies 9% less area than the Strassen based circuit.

The area occupation results are reported in the log-log plane in Fig. 3 where the trends lines are also reported. With the proposed implementation, the trend lines of silicon area occupation for the naive, Strassen based, and the AE implementations are:

$$\begin{aligned}
 A_{\text{Naive}} [\text{mm}^2] &\approx 2.6 \times 10^{-4} n^{1.95} \\
 A_{\text{Strassen}} [\text{mm}^2] &\approx 7.4 \times 10^{-4} n^{1.61} \\
 A_{\text{AlphaEvolve}} [\text{mm}^2] &\approx 2.7 \times 10^{-3} n^{1.17}
 \end{aligned} \tag{16}$$

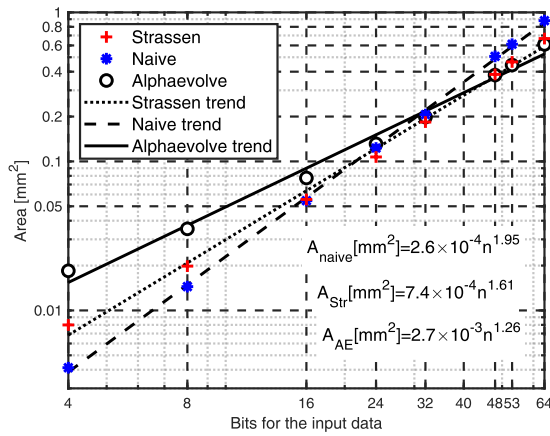


FIGURE 3: Area utilization of the proposed circuit implementation of the AlphaEvolve algorithm compared with the reference naive and Strassen based implementations as a function of the input bit width. Trend lines in the log-log plane and their corresponding equations are also included.

where n is the number of bits of the elements of the input matrices. Assuming that the designs keep following Eq. 16, at 128 bit the AE and the naive implementations should require an area occupation of 1.28 mm^2 and 3.34 mm^2 , respectively.

A similar trend is observed for the delay. At 4 bits, the AE design is more than three times slower than the naive implementation, but the gap decreases as the bit width increases. At 32 bits, the difference shrinks to a 34% increase, and at 64 bits the AE implementation is only 17% slower.

When compared with the Strassen based circuit the AE implementation is still slower but with a significantly reduced gap. While at 8 bit the delay of the Strassen based circuit is half of the AE implementation, at 48 bit the gap is 10% and at 64 bit the gap is 0.42 ns with both designs that are now bounded by the timing constraint of 10 ns.

However, it must be noted that the results for 48, 53, and 64 bits are influenced by the 10 ns synthesis constraint. The AE design reaches this limit at smaller bit widths than the naive design (for example, at 48 bits the AE design reaches 9.08 ns, whereas the naive implementation achieves 6.57 ns and is therefore still far from the 10 ns bound). As expected, the beneficial effect of the reduced number of multipliers in the AE implementation is less pronounced on delay than on area, because multiplier delay scales approximately linearly with bit width. Consequently, the additional levels of logic required to reduce the number of multipliers in the AE architecture have a negative impact on the critical path. This effect could be mitigated by implementing the multi-operand adder trees (A), (B), and (C) shown in Fig. 1 using architectures specifically optimized to reduce the number of logic levels in series.

Power dissipation, shown in Fig. 4 in the log-log plane, follows a similar pattern, though with a larger gap. With 32 bit inputs, the AE circuit consumes more than twice the power of the naive design. The gap decreases for larger bit width and at 54 and 64 bits the power dissipation tends to be similar to the

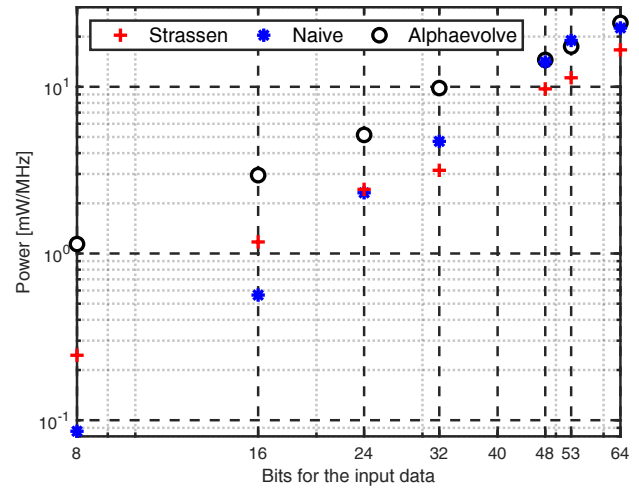


FIGURE 4: Log-log plot of the power dissipation for the proposed circuit implementation of the AlphaEvolve algorithm compared with the naive implementation. Results are shown as a function of the input bit width.

naive implementation. The Strassen based implementation, likely due to the combined advantages of smaller area and a more regular computational architecture, outperforms both the naive implementation and the AE circuit in terms of power dissipation when the bit width is 32 bits or larger. For bit widths below 32 bits, the Strassen based implementation is still more power efficient than the AE circuit, but the naive implementation exhibits the lowest power consumption.

VIII. CONCLUSION

This paper has presented, to the best of the author's knowledge, the first circuit level implementation of the groundbreaking Rank 48 AlphaEvolve algorithm for complex valued 4×4 matrix multiplication.

The proposed circuit is designed for full-precision computation and has been optimized for a fully parallel combinational implementation. Several optimization phases were carried out: a dedicated common subexpression elimination algorithm was employed to minimize the number of addition and subtraction operations; the complex multipliers were implemented using the Katsabura algorithm; and the bit widths of the internal signals were determined through a simulation based approach.

The resulting circuit has been open sourced together with the testbench, test vectors, and comparison circuit.

Implementation in 14 nm FinFET standard cell technology, with element bit widths ranging from 4 to 64 bits, show that the proposed implementation of AlphaEvolve algorithm requires fewer logic resources than the naive implementation for bit widths of 32 or larger. It also outperforms a Strassen based implementation, in terms of logic resource occupation, for bit widths of 48 or larger. At 64 bits the AlphaEvolve implementation require 31% less area than the naive design and 9% less area than the Strassen based implementation.

The advantage in terms of power dissipation is limited and only becomes apparent for designs with bit widths of

53 or more when compared against the naive design. In terms of speed the naive implementation outperforms both the proposed implementation the AlphaEvolve algorithm and the Strassen based implementation for the entire range of bit widths considered in this paper.

As a result of the findings of the paper, that suggest that AlphaEvolve implementation becomes advantageous in high-precision scenarios (32-bit to 64-bit), examples of practical deployment could be the scientific computing and the DSPs where 64-bit floating-point or high-precision fixed-point arithmetic is mandatory, and the edge devices for AI where silicon footprint is important to reduce cost and throughput is less relevant.

APPENDIX. BIT WIDTH DERIVATION FOR COMPLEX MULTIPLICATION

Assume all inputs are n bit signed 2's complement integers whose range is:

$$x \in [-2^{n-1}, 2^{n-1} - 1].$$

For a complex product $(a + jb)(c + jd)$ real and imaginary parts are given by:

$$\Re = ac - bd, \quad \Im = ad + bc.$$

For the real part, the maximum occurs when $a = b = c = -2^{n-1}$ and $d = 2^{n-1} - 1$ while the minimum occurs when $a = b = d = -2^{n-1}$ and $c = 2^{n-1} - 1$. With these value the resulting extrema are:

$$\Re_{\max} = 2^{2n-1} - 2^{n-1}, \quad \Re_{\min} = -2^{2n-1} + 2^{n-1},$$

Summing four products as required by a 4×4 matrix multiplication yields:

$$\Re_{\text{sum}} \in [-2^{2n+1} + 2^{n+1}, 2^{2n+1} - 2^{n+1}],$$

which requires a $(2n+2)$ bits 2's complement representation.

For the imaginary part, the maximum occurs when all terms are -2^{n-1} , while the minimum occurs when $a = c = -2^{n-1}$ and $b = d = 2^{n-1} - 1$. With these value the resulting extrema are:

$$\Im_{\max} = 2^{2n-1}, \quad \Im_{\min} = -2^{2n-1} + 2^n,$$

Summing four products as required by a 4×4 matrix multiplication yields:

$$\Im_{\text{sum}} \in [-2^{2n+1} + 2^{n+2}, 2^{2n+1}].$$

Note that in the worst case, the positive bound reaches exactly 2^{2n+1} , requiring a $2n + 3$ bits 2's complement representation.

For uniformity and to meet the worst case scenario, in the paper both real and imaginary outputs are represented on $(2n + 3)$ bits.

ACKNOWLEDGMENT

The author would like to thank Prof. A.G.M. Strollo for carefully proofreading the manuscript and for helpful comments.

REFERENCES

- [1] J. Dongarra and P. Luszczek, "Matrix multiplication on digital signal processors and hierarchical memory systems," in High Performance Computing. Springer, 1998, pp. 333–343.
- [2] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based accelerator design for deep convolutional neural networks," Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays, pp. 161–170, 2015.
- [3] X. Liu, Y. Chen, C. Hao, A. Dhar, and D. Chen, "WinoCNN: Kernel sharing Winograd systolic array for efficient convolutional neural network acceleration on FPGAs," in Proc. IEEE 32nd Int. Conf. Application-Specific Systems, Architectures and Processors (ASAP), 2021, pp. 258–265.
- [4] B. George, O. J. Omer, Z. Choudhury, A. V. and S. Subramoney, "A unified programmable edge matrix processor for deep neural networks and matrix algebra," ACM Trans. Embed. Comput. Syst., vol. 21, no. 5, Oct. 2022. [Online]. Available: <https://doi.org/10.1145/3524453>
- [5] E. Taka, A. Arora, K.-C. Wu, and D. Marculescu, "MaxEVA: Maximizing the efficiency of matrix multiplication on Versal AI engine," in 2023 International Conference on Field Programmable Technology (ICFPT), 2023, pp. 96–105.
- [6] C. Lee, H. Hasegawa, and S. Gao, "Complex-valued neural networks: A comprehensive survey," IEEE/CAA Journal of Automatica Sinica, vol. 9, no. 8, pp. 1406–1425, 2022. [Online]. Available: <https://www.ieee-jas.net/en/article/doi/10.1109/JAS.2022.105743>
- [7] C. Fang, S. Guo, W. Wu, J. Lin, Z. Wang, M.-K. Hsu, and L. Liu, "An efficient hardware accelerator for sparse transformer neural networks," in 2022 IEEE International Symposium on Circuits and Systems (ISCAS), 2022, pp. 2670–2674.
- [8] C. Du, S.-B. Ko, and H. Zhang, "Energy efficient fpga-based binary transformer accelerator for edge devices," in 2024 IEEE International Symposium on Circuits and Systems (ISCAS), 2024, pp. 1–5.
- [9] M. Zhao, W. Mao, and Z. Wang, "DATA: A memory-efficient deformable transformer accelerator via neural architecture search," IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 33, no. 11, pp. 3097–3108, 2025.
- [10] S. Li, Y. Kimura, H. Sato, and M. Fujita, "Parallelizing quantum simulation with decision diagrams," IEEE Transactions on Quantum Engineering, vol. 5, no. 01, pp. 1–12, Jan. 2024. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/TQE.2024.3364546>
- [11] A. M. Tulino and S. Verdú, "Random matrix theory and wireless communications," Foundations and Trends in Communications and Information Theory, vol. 1, no. 1, pp. 1–182, 2004.
- [12] H. Kim and W. J. Song, "LAS: Locality-aware scheduling for GEMM-accelerated convolutions in GPUs," IEEE Trans. Parallel Distrib. Syst., vol. 34, no. 5, pp. 1479–1494, 2023.
- [13] C. Wei, H. Jia, Y. Zhang, J. Yao, C. Li, and W. Cao, "IrGEMM: An input-aware tuning framework for irregular GEMM on ARM and X86 CPUs," IEEE Trans. Parallel Distrib. Syst., vol. 35, no. 9, pp. 1672–1689, 2024.
- [14] S. Lin, G. Xiao, H. Wang, W. Yang, K. Li, and K. Li, "High performance OpenCL-based GEMM kernel auto-tuned by bayesian optimization," IEEE Trans. Parallel Distrib. Syst., vol. 36, no. 9, pp. 1985–1997, 2025.
- [15] A. Petropoulos and T. Antonakopoulos, "A scalable FPGA architecture with adaptive memory utilization for GEMM-based operations," IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 33, no. 8, pp. 2334–2338, 2025.
- [16] V. Strassen, "Gaussian elimination is not optimal," Numerische Mathematik, vol. 13, pp. 354–356, 1969. [Online]. Available: <https://api.semanticscholar.org/CorpusID:121656251>
- [17] S. Winograd, "On multiplication of 2×2 matrices," Linear Algebra and Its Applications, vol. 4, no. 4, pp. 381–388, Oct. 1971.
- [18] E. Karstadt and O. Schwartz, "Matrix multiplication, a little faster," in Proc. ACM Symp. Parallelism Algorithms Architectures, Jul. 2017, pp. 101–110.
- [19] O. Schwartz and N. Vaknin, "Pebbling game and alternative basis for high performance matrix multiplication," SIAM J. Sci. Comput., vol. 45, no. 6, pp. C277–C303, 2023.
- [20] P.-W. Lai et al., "Accelerating Strassen-Winograd's matrix multiplication algorithm on GPUs," in Proc. IEEE Int. Conf. High Performance Computing, 2013, pp. 139–148.
- [21] A. G. Krishnan and D. Goswami, "Multi-stage memory efficient Strassen's matrix multiplication on GPU," in Proc. IEEE Int. Conf. High Performance Computing, Data, and Analytics (HiPC), 2021, pp. 212–221.

- [22] C. Misra et al., “Stark: Fast and scalable Strassen’s matrix multiplication using apache spark,” *IEEE Trans. Comput.*, vol. 8, no. 3, pp. 699–710, 2022.
- [23] H. M. D. Ip et al., “Strassen’s matrix multiplication for customisable processors,” in *Proc. 2002 IEEE Int. Conf. Field-Programmable Technology (FPT)*, Hong Kong, China, 2002, pp. 453–456.
- [24] T. E. Pogue and N. Nicolici, “Fast inner-product algorithms and architectures for deep neural network accelerators,” *IEEE Trans. Comput.*, vol. 73, no. 2, pp. 495–509, 2024.
- [25] —, “Strassen multisystolic array hardware architectures,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 5, pp. 1323–1333, May 2025.
- [26] A. Novikov, N. Vü, M. Eisenberger, E. Dupont, and P.-S. H. et al., “AlphaEvolve: A coding agent for scientific and algorithmic discovery,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.13131>
- [27] G. DeepMind, “AlphaEvolve: Mathematical results,” 2025. [Online]. Available: https://colab.research.google.com/github/google-deepmind/alphaevolve_results/blob/master/mathematical_results.ipynb
- [28] F. Bakreski, “AlphaEvolve-MatrixMul-Verification,” 2025, accessed: 2025-08-10. [Online]. Available: <https://github.com/PhialsBasement/AlphaEvolve-MatrixMul-Verification>
- [29] E. Napoli. (2025) Hardware-implementation-AlphaEvolve-MatrixMul. Accessed: Aug. 12, 2025, Commit: a1b2c3d. [Online]. Available: <https://github.com/etnapoli/Hardware-implementation-AlphaEvolve-MatrixMul>

APPENDIX. BIOGRAPHY SECTION



ETTORE NAPOLI (Senior Member, IEEE) received the Electronic Engineering degree (Hons.), in 1995, the Ph.D. degree in electronic engineering, in 1999, and the Physics degree (Hons.), in 2009. He was a Research Associate with the Engineering Department, University of Cambridge, U.K., in 2004. He was a Full Professor with the University of Naples Federico II, in 2020, and has been a Full Professor with the University of Salerno, since 2021. He has authored or co-authored more than a hundred papers published in international journals and conferences. His research interests include VLSI circuit design and modeling and design of power semiconductor devices.

...